

# Ejercicio 1

¿qué imprime el siguiente código?

```
class Demo:
    def __init__(self):
        self._x = 0
        self._interno = 0

    @property
    def x(self):
        return self._x

    @x.setter
    def x(self, value):
        self._x = value
        self._interno = 0 if self._x <= 0 else self._x * 2

    def __str__(self):
        return f'El valor de x es {self._x} y el valor interno es {self._interno}'
```

```
obj = Demo()
obj.x = 10
print(obj)
```

Seleccione una:

- ☐ a. No imprime nada. El código da error en la asignación, porque el método setter solo debería modificar el valor de la variable **\_x**.
- ☐ b. No imprime nada: da error, ya que hay dos métodos que se llaman igual.
- ☒ c. Imprime: El valor de x es 10 y el valor interno es 20  Correcto. Al asignar el valor 10, se invoca al método setter que actualiza el valor de **\_x** y de **\_interno**, por lo que al hacer el print toma el nuevo valor.
- ☐ d. Ninguna de las otras opciones mostradas.



# Ejercicio 2

¿qué imprime el siguiente código?

```
class Vehículo:
    def __init__(self, cantidad_puertas, cantidad_ruedas, motor='Diesel'):
        self.cant_puertas = cantidad_puertas
        self.cant_ruedas = cantidad_ruedas
        self.motor = motor
        # Identificador interno de fábrica (Mangling)
        self.__id_interno = "FAB-12345"

    def __str__(self):
        return f'Motor: {self.motor}, puertas: {self.cant_puertas}, ruedas: {self.cant_ruedas}'

class Transporte():
    def __init__(self, max_pasajeros, tipo='Público'):
        self.tipo = tipo
        self.cant_max_pasajero = max_pasajeros
        # Identificador interno administrativo (Mangling)
        self.__id_interno = "ADM-99999"

    def __str__(self):
        return f'Tipo de transporte: {self.tipo}'

class Automóvil(Vehículo, Transporte):
    def __init__(self, marca, modelo, patente, tipo='Particular', pasajeros=0, puertas=4, ruedas=4, motor='Nafta'):
        Vehículo.__init__(self, puertas, ruedas, motor)
        Transporte.__init__(self, pasajeros, tipo)
        self.marca = marca
        self.modelo = modelo
        self.patente = patente
```

```
auto = Automóvil("Toyota", "Corolla", "ABC-123")
```

```
print("Atributos del Automóvil:")
print(f"ID de auto: {auto.__id_interno}")
print(f"ID de Fábrica: {auto.Vehículo__id_interno}")
print(f"ID Administrativo: {auto.Transporte__id_interno}")
```

Seleccione una:

- ☐ a. Imprime: Atributos del Automóvil:  
ID de auto: ADM-99999  
ID de Fábrica: ADM-99999  
ID Administrativo: ADM-99999
- ☒ b. Imprime: Atributos del Automóvil:  Incorrecto. Python realiza una sustitución del nombre **\_\_id\_interno**, con lo cual **auto.\_\_id\_interno** no existe, por lo tanto da error al ejecutar `print(f"ID de auto: {auto.__id_interno}")`
- ☐ c. Imprime: "Atributos del Automóvil:" y luego da error ya que Python realiza una alteración del nombre de la variable `__id_interno` con lo cual `__id_interno` no se conoce.
- ☐ d. Imprime: Atributos del Automóvil:  
ID de auto: ADM-99999  
ID de Fábrica: FAB-12345  
ID Administrativo: ADM-99999

Respuesta incorrecta.

La respuesta correcta es:

Imprime: "Atributos del Automóvil:" y luego da error ya que Python realiza una alteración del nombre de la variable `__id_interno` con lo cual `__id_interno` no se conoce.



# Ejercicio 3

¿qué imprime el siguiente código?

```
class A():
    def __init__(self):
        self.a = "a de A"

class B():
    def __init__(self):
        self.a = "a de B"

class AB(B, A):
    def __init__(self):
        A.__init__(self)
        B.__init__(self)
        self.c = "c de AB"

    def __str__(self):
        return f'a = {self.a}'

obj = AB()
print(obj)
```

Seleccione una:

- ☐ a. Ninguna de las otras opciones mostradas.
- ☐ b. No imprime nada: da error porque primero se deberían definir las variables de instancia y luego invocar a los `__init__` de A y de B.
- ☐ c. Imprime: a = a de A
- ☒ d. Imprime: a = a de B  Correcta. Cuando invocás al `__init__` de B se sobrescribe la variable a. definida en `__init__` de A.

Respuesta correcta

La respuesta correcta es:

Imprime: a = a de B

## Ejercicio 4

¿qué imprime el siguiente código?

```
class A:
    def __init__(self, value=None):
        self._x = value==None

    @property
    def x(self):
        return self._x

    def __str__(self):
        return f"El valor de x es {self.x}"

obj = A()
print(obj)
```

Seleccione una:

- ☐ a. Ninguna de las otras opciones mostradas.
- ☐ b. No imprime nada: da error, ya que no está definido el método setter, con lo cual no se puede asignar ningún valor a x.
- ☐ c. Imprime: el valor de x es None
- ☒ d. Imprime: el valor de x es True Correcto. Al crear el objeto, se invoca el método `__init__` y ahí se asigna el valor de la variable x en función del parámetro pasado. En ese caso, al no pasar ningún parámetro asume None, con lo cual la comparación da True.
- ☐ e. Imprime: el valor de x es False

Respuesta correcta

La respuesta correcta es:

Imprime: el valor de x es True

# Ejercicio 5

¿qué imprime el siguiente código?

```
class SuperHero():  
  
    def __init__(self, real_name, name):  
        self._real_name = real_name  
        self.name = name  
        self.enemies = []  
  
batman = SuperHero("Bruce Wayne ", "Batman")  
print(batman.name)
```

Seleccione una:

- ☒ a. Imprime: Batman.  Correcta. En este caso estás accediendo a una variable de instancia pública, por lo que no es necesario utilizar un método especial para acceder a la misma.
- ☐ b. Ninguna de las otras respuestas mostradas.
- ☐ c. Imprime: algo similar a <\_\_main\_\_.SuperHero object at 0x....0>
- ☐ d. No imprime nada: el código da error ya que nunca se invocó al método `__init__`, donde se crea la variable de instancia **name**.
- ☐ e. No imprime nada: el código es erróneo ya que no se puede imprimir la variable de instancia **name** sin utilizar el método `get` correspondiente o generando una propiedad.

Respuesta correcta

La respuesta correcta es:

Imprime: Batman.